Model Checking for Knowing-how Logics

Rens Doodeman, Ziqi Wang, and Zixuan Chen

Monday 23rd March, 2026

Abstract

We implement model checkers in Haskell for both the basic knowing-how logic $\mathcal{L}_{Kh}$ and the uncertainty-based knowing-how logic with regularity constraints reg-$\mathcal{L}^{U}_{KH}$. For both logics, we validate the implementation by means of a range of tests using QuickCheck. In addition, to improve usability, we provide a parser for each logic.

Contents

1 Introduction **3**

2 Basic Knowing How Logic $\mathcal{L}_{Kh}$ **3**

2.1 Preliminaries 3

2.2 Haskell Representation 4

2.3 Model checker in Haskell 6

2.4 Parsing for $\mathcal{L}_{Kh}$ 6

2.5 Random Generation for $\mathcal{L}_{Kh}$ 7

2.5.1 Model Generation with Parameters 7

2.5.2 Arbitrary Instances for QuickCheck 9

3 Uncertainty-based Knowing-how logic with Regularity Constraints reg-$\mathcal{L}^{U}_{KH}$ **10**

3.1 Preliminaries 10

3.2 Haskell Representation 12

3.3 Model checker in Haskell 14

3.4 Parsing for reg-$\mathcal{L}_{KH}^{U}$ 18
3.5 Random Generation for reg-$\mathcal{L}_{KH}^{U}$ 19
3.5.1 Model Generation with Parameters 19
3.5.2 Arbitrary Instances for QuickCheck 21

4 Testing and Validation **22**
4.1 Tests for $\mathcal{L}_{Kh}$ 22
4.2 Tests for reg-$\mathcal{L}_{KH}^{U}$ 23
4.3 HSpec Test Suite 23

5 Conclusion **25**

Bibliography **25**

1 Introduction

Ability-based epistemic logics are found many places in computer science, as arguably they provide us with a formal approach for strategic reasoning and AI planning. Several logics in this family have been studied lately. In particular, epistemic logics describing the notion of *knowing how* received a lot of attention, due to their simplicity and good meta-logical properties.

The simplest version of *knowing how*, $\mathcal{L}_{Kh}$, based on the paper by (Wang 2015[3]), with the semantics reformulated in (Areces et al. 2021[1]). This logic captures the agent's ability to guarantee the completion of a task under a given condition. This idea is ideal in a sense, since even though the agent manages to do something under a condition, it does not imply the agent really know how. For example, one can win a lottery does not imply one can guarantee how to win. The uncertainty is involved in this case. Therefore, another logic may be preferred, $\mathcal{L}^U_{Kh}$. In this logic, an agent may have *indistinguishability* among several plans they might take, a critical concept in epistemic logic. Indistinguishable plans form equivalence classes, partitioning the set of available plans. Intuitively, plans considered identical by the agent are placed in the same class. Furthermore, this logic involves multi-agents, as opposed to a single agent. Finally, a new logic, reg-$\mathcal{L}^U_{Kh}$, a variation of the second version, introduces equivalence classes that are defined in terms of finite automatons.

In this project, we modelled a family of knowing-how logics and implemented model checkers for both $\mathcal{L}_{Kh}$ and reg-$\mathcal{L}^U_{Kh}$. The checker for $\mathcal{L}_{Kh}$ is an explicit procedure with bounded plan length, while the checker for reg-$\mathcal{L}^U_{Kh}$ follows the algorithms described in [2]. In addition, we implemented parsers for both logics and used QuickCheck to test several basic properties of the resulting systems. Together, these form the main contributions of the project.

2 Basic Knowing How Logic $\mathcal{L}_{Kh}$

In this section we model the language of *basic knowing how* $\mathcal{L}_{Kh}$ introduced in [3]. This logic captures the ability of an agent to achieve a goal φ given a precondition ψ, by requiring the existence of a plan that is strongly executable and goal-reaching from every state satisfying ψ. The underlying models are labelled transition systems, where each relation corresponds to an action the agent can perform.

We implement an explicit model-checker for this logic with a bounded plan depth, along with QuickCheck generators for random testing of the semantics.

2.1 Preliminaries

Definition 2.1. (Syntax) Given a set of proposition letters $Prop$, the language $\mathcal{L}_{Kh}$ is defined by

$$\varphi ::= \top \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid Kh(\varphi, \varphi),$$

where $p \in Prop$.

Definition 2.2 (LTS)**.** A labelled transition system (LTS) is a tuple $\mathcal{M} = (S, (R_a)_{a \in Act}, V)$, where:

- S is a non-empty set of states,
- $(R_a)_{a \in Act}$ is a family of binary relations on S, each labelled by an action $a \in Act$,
- $V : S \to 2^{Prop}$ is a valuation function.

We write $s \xrightarrow{a} t$ if $(s, t) \in R_a$. For a plan $\sigma = a_1 \dots a_n \in Act^*$, we write $s \xrightarrow{\sigma} t$ if there exist $s_1, \dots, s_{n-1}$ such that $s \xrightarrow{a_1} s_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} t$. A plan $\sigma = a_1 \dots a_n$ is *strongly executable* at s iff for every $0 \leq k < n$ and every t such that $s \xrightarrow{\sigma_k} t$, the state t has at least one a_{k+1}-successor.

Definition 2.3 (Semantics)**.** Let $\mathcal{M}$ be an LTS, and s be a state.

$\mathcal{M}, s \models \top$	*iff*	*always*
$\mathcal{M}, s \models p$	*iff*	$p \in V(s)$
$\mathcal{M}, s \models \neg\varphi$	*iff*	$\mathcal{M}, s \not\models \varphi$
$\mathcal{M}, s \models \varphi \wedge \psi$	*iff*	$\mathcal{M}, s \models \varphi$ and $\mathcal{M}, s \models \psi$
$\mathcal{M}, s \models Kh(\psi, \varphi)$	*iff*	there exists a $\sigma \in Act^*$ such that for all s' such that $\mathcal{M}, s' \models \psi$: σ is strongly executable at s' and for all t such that $s' \xrightarrow{\sigma} t$, $\mathcal{M}, t \models \varphi$

The $Kh(\psi, \varphi)$ can be interpreted as "the agent knows how to achieve φ given ψ". Notice that the Kh is a global modality. Namely, either it holds for all states, or none of them.

2.2 Haskell Representation

The syntax is modelled following Definition 3.1.

```
module SingleAgent where

import Data.List (nub, delete)
-- import NoneEmpty including its constructor :|
import Data.List.NonEmpty (NonEmpty(..), toList)
-- import parsec but hide State to avoid conflicts
import Text.Parsec hiding (State)
import Test.QuickCheck

type Proposition = Int

data Form = P Proposition | Neg Form | Conj Form Form | KH Form Form | T
    deriving (Eq, Show, Ord)
```

Following Definition 3.2, we model the LTS as follows. When creating a LTS, consider that states :: NonEmpty State, and therefore must use the :| constructor, i.e. (n :| [n+1..]). For more information see NonEmpty on Hoogle.

```
type Action = Integer
type Plan = [Action]
type State = Int
type Valuation = [(State, [Proposition])]
```

```haskell
data AbilityMap = LTS {
    states :: NonEmpty State,
    transitions :: Relations,
    valuation :: Valuation
} deriving(Show)
```

The relations and strong executability in Definition 3.2 are implemented below. Both of these are binary relations on states. Therefore, in our implementation, we use the same Haskell type `Rel` to represent them, since both can be understood as sets of pairs of states.

To represent the family of atomic relations $R = (R_a)_a$, we index relations by actions. This gives rise to the type `Relations`, corresponding to the collection of action-labelled transitions in the literature.

We do not model R_σ explicitly as a separate data structure. Instead, we only implement the operations needed to reason about plan execution and strong executability.

For this purpose, we define the following helper functions:

- `image`, which computes the image of a state under a relation, i.e. it collects all successor states of a given state;
- `r_a`, which takes an action a and the family of relations R, and returns the indexed relation R_a;
- `executePlan`, which computes all possible end states obtained after executing a plan from a given initial state;
- `stronglyExecutableAt`, which determines whether a plan is strongly executable at a given state.

```haskell
-- Both $R_a$ and $R_\sigma$ share the same type.
-- Here we model the whole set of relations, without classification.
type Rel = [(State, State)]

-- The family of action-indexed relations (R_a)_a
type Relations = [(Action, Rel)]

-- Image of a state under R_a
image :: Rel -> State -> [State]
image r u = [v | (x, v) <- r, x == u]

-- Given (R_a)_a and an action x, return R_x
r_a :: Relations -> Action -> Rel
r_a rs a =
    case lookup a rs of
        Just r  -> r
        Nothing -> []

-- Execute a plan from a given start state
-- This corresponds to R_sigma(u)
executePlan :: Relations -> State -> Plan -> [State]
executePlan _  u []       = [u]
executePlan rs u (a:sigma) =
    nub (concat [ executePlan rs v sigma | v <- image (r_a rs a) u ])

-- Plan is SE or not at a state
stronglyExecutableAt :: Relations -> State -> Plan -> Bool
stronglyExecutableAt _  _ []       = True
stronglyExecutableAt rs u (a:sigma) =
    let next = image (r_a rs a) u
```

```
    in  not (null next) &&
        all (\v -> stronglyExecutableAt rs v sigma) next
```

2.3 Model checker in Haskell

We now implement the semantics in Definition 3.3. Note that `findPlans` enumerates all plans up to a fixed depth (currently 5). This is a bounded approximation: if a witness plan longer than the bound exists, the checker will not find it. A complete decision procedure would require another PSPACE-Complete algorithm in Theorem 1 from [2].

```
-- Given an LTS and formula, returns the states that satisfy said formula
statesSatisifying :: AbilityMap -> Form -> [State]
statesSatisifying m f = [s | s <- toList (states m), isTrue (m, s) f]

-- Given an LTS, find all plans.
findPlans :: AbilityMap -> [Plan]
findPlans m = nub (concatMap (plansFrom depth) (toList (states m)))
  where
    -- For now we limit the depth to avoid infinite loops.
    depth = 5

    -- Flatten transitions
    edges :: [(Action,(State,State))]
    edges = [ (a,(u,v))| (a,rel) <- transitions m, (u,v) <- rel ]

    -- Generate all plans from a given state up to our specified depth
    plansFrom :: Int -> State -> [Plan]
    plansFrom 0 _ = []
    plansFrom d s =
        [ [a] | (a,(x,_)) <- edges, x == s ] -- single action plans
        ++ [ a:p | (a,(x,y)) <- edges, x == s, p <- plansFrom (d-1) y ]

isTrue :: (AbilityMap, State) -> Form -> Bool
isTrue _ T = True
isTrue (m, s) (P p) =
  case lookup s (valuation m) of
    Just props -> p `elem` props
    Nothing -> False
isTrue (m, s) (Neg f) = not (isTrue (m, s) f)
isTrue (m, s) (Conj f g) = isTrue (m, s) f && isTrue (m, s) g
-- KH is NOT local; its truth does not depend on the state at which it is evaluated.
-- KH either holds at all states, or none of them.
isTrue (m, _) (KH f g) =
    any (\a ->
        all (\s ->
            stronglyExecutableAt rs s a
            && all (\t -> isTrue (m,t) g) (executePlan rs s a)
        ) statesF
    ) candidatePlans
  where
    rs = transitions m
    statesF = statesSatisifying m f
    candidatePlans = findPlans m

-- Infix alias for the satisfaction relation
(|=) :: (AbilityMap, State) -> Form -> Bool
(|=) = isTrue
```

2.4 Parsing for $\mathcal{L}_{Kh}$

Formulas may be created in ghci using `parseForm`. The following inputs are accepted.

- 'p' or 'P' followed by an integer n returns `P n`
- 'T' returns `T`
- '!' followed by a valid input p returns `Neg p`
- '^' prefixed and followed by valid inputs p and q returns `Conj p q`
- "KH" followed by valid inputs p and q returns `KH p q`
- "->" prefixed and followed by valid inputs p and q returns `Neg (Conj p (Neg q))` (abbreviation)
- 'v' or 'V' prefixed and followed by valid inputs p and q returns `Neg (Conj p (Neg q))` (abbreviation)

```
pForm :: Parsec String () Form
pForm = spaces *> pImpl where
    -- Start parsing at implication level
    -- Abbreviation: Implication (right-associative)
    pImpl = chainr1 pDisj (spaces *> string "->" *> spaces >> return (\p q -> Neg (Conj p (Neg q))))

    -- Abbreviation: Disjunction (left-associative)
    pDisj = chainl1 pConj (spaces *> oneOf "vV" *> spaces >> return (\p q -> Neg (Conj (Neg p) (Neg
        q))))

    -- Conjunction (left-associative)
    pConj = chainl1 pPrefix (spaces *> char '^' *> spaces >> return Conj)

    pPrefix = try pNeg <|> try pKH <|> pTrue <|> pRemainder

    -- Negation
    pNeg = char '!' >> spaces >> Neg <$> pPrefix

    -- Knowing How
    pKH = try (KH <$> (string "KH" >> spaces *> pPrefix)
        <*> (spaces *> pPrefix))

    -- Truth-constant
    pTrue = char 'T' >> return T

    pRemainder = pVar <|> between (char '(' *> spaces) (spaces *> char ')') pForm
    pVar = spaces *> oneOf "pP" *> spaces *> (P . read <$> many1 digit) <* spaces


parseForm :: String -> Either ParseError Form
parseForm = parse (pForm <* eof) "input"

evalForm :: (AbilityMap, State) -> String -> Bool
evalForm (m, s) str =
    case parseForm str of
        Right f -> isTrue (m, s) f
        Left _  -> error "Invalid formula"
```

2.5 Random Generation for $\mathcal{L}_{Kh}$

2.5.1 Model Generation with Parameters

To facilitate testing with models of specific sizes, we implement a function that generates an *AbilityMap* from a fixed number of states, propositions, and actions.

The generated model includes:

- a finite set of states $\{1, \ldots, n\}$,
- a set of actions $\{1, \ldots, k\}$,
- a valuation assigning each state a random subset of propositions $\{1, \ldots, m\}$,
- a family of action-labelled transitions, each generated by randomly sampling pairs of states.

```
-- Generate an LTS with a specific number of states, propositions, and actions.
-- n: number of states
-- m: number of propositions
-- k: number of actions
generateLTS :: Int -> Int -> Int -> Gen AbilityMap
generateLTS n m k = do
    -- Ensure that the model has at least one state
    let n' = max 1 n
    let m' = max 0 m
    let k' = max 0 k

    let sts = [1 .. n']
    let stateList = head sts :| tail sts

    -- Available actions and propositions
    let actions = [1 .. fromIntegral k']
    let propRange = [1 .. m']

    -- Generate one random relation for each action
    rels <- mapM (\_ -> generateRandomRel sts) actions

    -- Generate one random valuation for each state
    vals <- mapM (generateRandomValuation propRange) sts

    return $ LTS stateList (zip actions rels) vals

-- Generate a random binary relation on the given set of states.
-- The relation is represented as a list of pairs (u,v).
generateRandomRel :: [State] -> Gen Rel
generateRandomRel sts = do
    -- Randomly decide how many edges to generate
    numEdges <- choose (0, length sts * length sts)

    -- Generate that many pairs of states
    edges <- vectorOf numEdges $ do
        u <- elements sts
        v <- elements sts
        return (u, v)

    -- Remove duplicates so that the relation behaves like a set
    return (nub edges)

-- Generate a random valuation for a state
-- by picking a random subset of the available propositions.
generateRandomValuation :: [Proposition] -> State -> Gen (State, [Proposition])
generateRandomValuation propRange s = do
    props <- sublistOf propRange
    return (s, props)
```

It is possible to generate concrete models and test formulas interactively in `ghci`. For instance:

```
ghci> m <- generate (generateLTS 5 3 2)
ghci> m
LTS {states = 1 :| [2,3,4,5],
transitions = [(1,[(4,5),(4,2),(4,3),(5,1),(1,4),(2,2),
(3,1),(4,4),(2,3)]), (2,[(4,3),(4,5),(2,5),(4,2),(5,4),(1,5)])],
```

```
valuation = [(1,[1,3]),(2,[2,3]),(3,[1,2,3]),(4,[2,3]),(5,[2])]}

-- Evaluate a formula at a state
ghci> isTrue (m, 1) (KH (P 1) (P 2))
True

-- Using the parser
ghci> evalForm (m, 1) "KH p1 p2"
True
```

In the example above, `generateLTS 5 3 2` produces a random LTS with five states, three proposition letters, and two actions. The function `isTrue` can then be used to evaluate formulas directly, while `evalForm` allows testing formulas given as strings.

2.5.2 Arbitrary Instances for QuickCheck

Finally, for this section we define the instances of Arbitrary for `Form` and `Arbitrary` respectively. For `Form` we use the `sized` function to ensure that the generated formulas remain finite in size.

```
instance Arbitrary Form where
    arbitrary = sized randomForm where
        -- Helper function to generate random formulas of a given size
        randomForm :: Int -> Gen Form
        randomForm 0 = oneof [P <$> choose (1, 5), return T]
        randomForm n = oneof
            [ P <$> choose (1, 5)
            , return T
            , Neg <$> randomForm (n - 1)
            , Conj <$> randomForm (n `div` 2) <*> randomForm (n `div` 2)
            , KH <$> randomForm (n `div` 2) <*> randomForm (n `div` 2)
            ]

instance Arbitrary AbilityMap where
    arbitrary = do
        n <- choose (1,10)
        let sts = [1..n] -- n states
        rels <- generateRelations n sts
        vals <- mapM generateValuation sts
        return $ LTS (head sts :| tail sts) rels vals
        where
            generateRelations n sts
                | n == 1 = return []
                | otherwise = do
                    m <- choose (1,n) -- decide how many actions to generate
                    actions <- vectorOf m (choose (1,5))
                    mapM (generateRel sts) actions

             -- for each action a, generate a relation labeled by a
            generateRel sts a = do
                x <- elements sts
                y <- elements (delete x sts) -- avoid loops
                return (a, [(x,y)])

            -- for each state s, generate a list of propositions
            generateValuation s = do
                props <- listOf (choose (1,5))
                return (s, nub props)
```

It is possible to try out semantics by running `stack ghci`. For example:

```
ghci> generate (arbitrary:: Gen AbilityMap)
LTS {states = 1 :| [2,3,4,5,6], ...
ghci> m = LTS {states = 1 :| [2,3,4,5,6], ...
ghci> isTrue (m, 2) (KH (P 4) (P 1))
False
ghci> evalForm (m, 2) "KH (p4 ^ !p2) p3"
True
```

3 Uncertainty-based Knowing-how logic with Regularity Constraints $reg\text{-}\mathcal{L}^U_{KH}$

In the framework of *basic knowing how* we introduced above, an agent possesses the ability to achieve the goal φ given ψ if and only if he has a plan that is fail-proof, meaning that each partial execution must be completable. In scenarios where the agent lacks this ability, it is only because a sequence of actions cannot be generated due to certain environmental constraints. However, another scenario may also occur: the agent does not know which plan is adequate for the situation. All he can do is blindly apply a plan he thought might work, which may or may not be successful. Such *indistinguishability* among plans establishes an epistemic relation of *knowing that*.

We now introduce a new logic of *knowing how* for a multi-agent setting defined in [2], extending the LTS with a notion of epistemic indistinguishability between plans. The uncertainty of an agent is encoded by equivalence classes of plans, and each such class is represented by a finite automaton. More precisely, each equivalence class of indistinguishable plans is represented by the language recognised by a finite automaton.

In Theorem 2 from [2], it is shown that the model-checking problem for this logic is in PTIME. The proof relies on reducing both conditions of the Kh_i-modality to graph reachability checks. In this section, we provide a concrete implementation of this algorithm in Haskell, closely following the constructions from the paper.

3.1 Preliminaries

Definition 3.1 (Syntax)**.** Given a set of proposition letters $Prop$ and a set of agents Agt, where $p \in Prop$ and $i \in Agt$, the language $reg\text{-}\mathcal{L}^U_{KH}$ is defined by

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid Kh_i(\psi, \varphi).$$

We first introduce the automata used to represent classes of plans.

Definition 3.2 (Automaton)**.** A non-deterministic finite automaton is a tuple $\mathcal{A} = (Q, Act, \delta, I, F)$, where:

- Q is a finite set of states,
- Act is a finite alphabet of actions,
- $\delta : Q \times Act \to 2^Q$ is a transition function,

- $I \subseteq Q$ is the set of initial states,
- $F \subseteq Q$ is the set of accepting (final) states.

To connect an automaton with the underlying transition system, we use the following product digraph.

Definition 3.3 (Digraph)**.** Given a reg-LTS^U $\mathcal{S}$ and an automaton $\mathcal{A} = (Q, Act, \delta, I, F)$, we define a digraph $G = (V, E)$ as follows:

- $V = Q \times S$, representing the combined states of the automaton and the underlying transition system;
- E is the set of edges such that $((q, t), (q', t')) \in E$ if and only if there exists an action $a \in Act$ such that:
  1. $q \xrightarrow{a} q'$ in $\mathcal{A}$;
  2. $(t, t') \in R_a$.

This construction synchronizes transitions of the automaton with transitions of the LTS under the same action, allowing us to track which runs of the automaton are realizable in the model.

We next recall the relational notions needed for the semantics.

Definition 3.4. For $\pi \subseteq Act^*$, $u \in S$, and $U \subseteq S$, define

$$R_\pi := \bigcup_{\sigma \in \pi} R_\sigma, \qquad R_\pi(u) := \bigcup_{\sigma \in \pi} R_\sigma(u), \qquad R_\pi(U) := \bigcup_{u \in U} R_\pi(u).$$

A set of plans $\pi \subseteq Act^*$ is *strongly executable* at $u \in S$ iff every plan $\sigma \in \pi$ is strongly executable at u. This gives rise to the definition $SE(\pi) := \bigcap_{\sigma \in \pi} SE(\sigma)$, which collects all states at which every plan in π is strongly executable.

These notions can be lifted from plan sets to automata via their accepted languages.

Definition 3.5. In terms of automata, $\mathrm{SE}(L(\mathcal{A})) := \bigcap_{\sigma \in L(\mathcal{A})} \mathrm{SE}(\sigma)$ collects all states at which every plan in $L(\mathcal{A})$ is strongly executable, and $R_{L(\mathcal{A})} := \bigcup_{\sigma \in L(\mathcal{A})} R_\sigma$ collects all transitions realisable by some plan in $L(\mathcal{A})$.

We now define reg-LTS^U models.

Definition 3.6. A reg-LTS^U is a tuple $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$, where the elements of U_i associated with an agent i are finite automata, namely,

$$\emptyset \neq U_i = \{\mathcal{A}_1, \mathcal{A}_2, \ldots\}.$$

Here, $L(\mathcal{A}_i)$ denotes the language accepted by $\mathcal{A}_i$. Moreover, for every $\mathcal{A}_m, \mathcal{A}_n \in U_i$ such that $m \neq n$, we have

$$L(\mathcal{A}_m) \cap L(\mathcal{A}_n) = \emptyset.$$

Finally, we can state the semantics of the language.

Definition 3.7 (Semantics). Let $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$ be a finite reg-LTS^U, and $s \in S$. The satisfaction relation $\models$ is defined as:

$\mathcal{S}, s \Vdash p$	*iff*	$p \in V(s)$
$\mathcal{S}, s \Vdash \neg\varphi$	*iff*	$\mathcal{S}, s \nVdash \varphi$
$\mathcal{S}, s \Vdash \psi \vee \varphi$	*iff*	$\mathcal{S}, s \Vdash \psi$ or $\mathcal{S}, s \Vdash \varphi$
$\mathcal{S}, s \Vdash Kh_i(\psi, \varphi)$	*iff*	there is a finite automaton $\mathcal{A} \in U_i$, such that (1) $[\![\psi]\!]^{\mathcal{S}} \subseteq SE(L(\mathcal{A}))$ and (2) for every $t \in [\![\psi]\!]^{\mathcal{S}}$, $R_{L(\mathcal{A})}(t) \subseteq [\![\varphi]\!]^{\mathcal{S}}$,

where $[\![\psi]\!]^{\mathcal{S}} := \{w \in S \mid \mathcal{S}, s \Vdash \psi\}$ is the set of all states satisfying ψ.

The $Kh_i(\psi, \varphi)$ can be interpreted as "when ψ is the case, the agent i knows how to make φ true". Kh_i is also a global modality.

3.2 Haskell Representation

The syntax is modelled following Definition 4.1.

```
module MultiAgent where

import SingleAgent
  ( Proposition
  , Action
  , Plan
  , State
  , Valuation
  , Rel
  , Relations
  , executePlan
  , stronglyExecutableAt
  , image
  , r_a
  )

import Data.List (nub)
import Data.Maybe (fromMaybe)
import Test.QuickCheck

-- import parsec but hide State to avoid conflicts
import Text.Parsec hiding (State)

type Agent = Int

data RegForm = Prop Proposition | Not RegForm | Disj RegForm RegForm | KHI Agent RegForm RegForm
    deriving (Eq, Show, Ord)
```

The automaton type implements Definition 4.2.

```
type Successors = [State]
type ActionAtState = (State, Action)

data Automaton = ATMN {
    statesA :: [State],
    actionsA :: [Action],
    transitionsA :: [(ActionAtState, Successors)],
    initial :: [State],
    final :: [State]
} deriving (Eq, Show, Ord)
```

The product digraph from Definition 4.3 is implemented below. The helper functions `getAutNext` and `getLtsNext` retrieve successor states in the automaton and LTS re-

spectively.

```
type GVertex = (State, State) -- (Automaton State, LTS State)
type GEdge = (GVertex, GVertex)

data Digraph = Digraph {
    vSet :: [GVertex],
    eSet :: [GEdge]
} deriving (Eq, Show, Ord)

-- Helper function to get next automaton states under a given action
getAutNext :: Automaton -> State -> Action -> [State]
getAutNext atmn q a =
    fromMaybe [] (lookup (q, a) (transitionsA atmn))

-- Helper function to get next LTS states under a given action
getLtsNext :: RegLTSU -> State -> Action -> [State]
getLtsNext m t a = image (r_a (relationsM m) a) t

-- Build the product digraph G = (V, E)
buildDigraph :: RegLTSU -> Automaton -> Digraph
buildDigraph m atmn = Digraph nodes edges
  where
    nodes = [(q, s) | q <- statesA atmn, s <- statesM m]
    edges = [ ((q, t), (q', t'))
            | (q, t) <- nodes
            , act    <- actionsA atmn
            , q'     <- getAutNext atmn q act
            , t'     <- getLtsNext m t act
            ]
```

The following functions implement the relational notions from Definition 4.4.

```
type PlanSet = [Plan]

-- R_pi(u) = union of R_sigma(u) for all sigma in pi
rPiU :: Relations -> State -> PlanSet -> [State]
rPiU rs u plans =
    nub (concat [executePlan rs u sigma | sigma <- plans])

-- R_pi(X) = union of R_pi(u) for all u in X
rPiX :: Relations -> [State] -> PlanSet -> [State]
rPiX rs xs plans =
    nub (concat [rPiU rs u plans | u <- xs])

-- SE(sigma) = set of all states at which sigma is strongly executable
seSigma :: [State] -> Relations -> Plan -> [State]
seSigma sts rs sigma =
    [u | u <- sts, stronglyExecutableAt rs u sigma]

-- SE(pi) = intersection of SE(sigma) for all sigma in pi
sePi :: [State] -> Relations -> PlanSet -> [State]
sePi sts rs plans =
    [u | u <- sts, all (stronglyExecutableAt rs u) plans]
```

The functions above are not used directly in the model-checking algorithm, but are included to align with the notions from the paper. For the same reason, we omit the implementation of definition 4.5. Because 4.4 and 4.5 are simply same concepts but in different perspectives.

Finally, the reg-LTS^U model implements Definition 4.6.

```
type Uncertainty = [(Agent, [Automaton])] -- U_i = {A_1,...}, for each agent i

data RegLTSU = RegLTSU{
        statesM :: [State],
```

```
        relationsM :: Relations,
        uncertainty :: Uncertainty,
        valuationM :: Valuation -- use the Valuation from singleAgent
    } deriving (Eq, Show, Ord)

-- Given an agent i, return U_i = [A_1,...]
getAgentAuts :: RegLTSU -> Agent -> [Automaton]
getAgentAuts m agent =
    fromMaybe [] (lookup agent (uncertainty m))
```

3.3 Model checker in Haskell

We now discuss the implementation model checking algorithms for $reg\text{-}\mathcal{L}^U_{KH}$ in Haskell.

Given a finite reg-LTS^U $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$, a state $s \in S$, and a formula $Kh_i(\varphi, \psi)$, we check whether $\mathcal{S}, s \models Kh_i(\varphi, \psi)$ by iterating over all automata $\mathcal{A} \in U_a$ and verifying two conditions:

1. $[\![\varphi]\!]^{\mathcal{S}} \subseteq \mathrm{SE}(L(\mathcal{A}))$
2. $R_{L(\mathcal{A})}([\![\varphi]\!]^{\mathcal{S}}) \subseteq [\![\psi]\!]^{\mathcal{S}}$

Once we find such automaton, the formula $Kh_i(\varphi, \psi)$ is then satisfied. Since $L(\mathcal{A})$ may be infinite, directly enumerating all plans is not feasible. Following [2], we handle each condition via a separate algorithm, both of which reduce the problem to reachability checks on finite graphs.

To perform the reachability checks in PTIME, we use the following *depth-first search* algorithm:

Require: Digraph $G = (V, E)$, start vertex v_0, target set $T_0 \subseteq V$
DFS($G, v_0, T_0, \emptyset$)
function DFS($G, v, T, visited$)
    if $v \in T$ **then return true**
    end if
    if $v \in visited$ **then return false**
    end if
    $visited \leftarrow visited \cup \{v\}$
    for $w \in$ successors(G, v) **do**
        if DFS($G, w, T, visited$) **then return true**
        end if
    end for
    return false
end function

The algorithm above runs in $O(|V| + |E|)$, which is polynomial. In Haskell:

```
-- Successors of a vertex in the digraph
successors :: Digraph -> GVertex -> [GVertex]
successors g v = [v' | (u, v') <- eSet g, u == v]

-- DFS reachability: is any vertex in targetSet reachable from start?
-- The type can be read as: digraph -> start -> targets -> visited_set -> Bool
dfsReachable :: Digraph -> GVertex -> [GVertex] -> [GVertex] -> Bool
dfsReachable _ start targetSet _
```

```
    | start `elem` targetSet = True
dfsReachable _ start _ visited
    | start `elem` visited = False
dfsReachable g start targetSet visited =
    any (\v -> dfsReachable g v targetSet (start : visited)) (successors g start)
```

The algorithm that checks Condition (1) from [2] works as follows:

Require: reg-LTS^U $\mathcal{S}$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$

function CheckSE-Original($\mathcal{S}, s, \mathcal{A}$)
  Build the product digraph $G = (Q \times S,\ E)$
  for $a \in Act$ **do**
    for $t \in S$ such that $R_a(t) = \emptyset$ **do**
      for $q \in Q$ such that $\delta(q, a) \neq \emptyset$ **do**
        for $q_0 \in I$ **do**
          if (q, t) is reachable from (q_0, s) in G **then**
            return true
          end if
        end for
      end for
    end for
  end for
  return false
end function

Instead of using the nested for-loops, we first collect all *bad vertices* via set comprehension, i.e.

$$Bad = \{(q, t) \in Q \times S \mid \exists\, a \in Act : \delta(q, a) \neq \emptyset \text{ and } R_a(t) = \emptyset\},$$

and then perform a single DFS reachability check from each initial vertex (q_0, s) to the set Bad. This is equivalent to the original procedure, but avoids repeated reachability checks.

A bad vertex (q, t) represents a situation where the automaton expects to perform some action a at state q, but the LTS cannot execute a at state t. If such a vertex is reachable, it means that some plan in $L(\mathcal{A})$ fails to be strongly executable.

Remark: This function might be more accurately called `checkNotSE` rather than `checkSE`, since it returns true exactly when strong executability fails.

Require: reg-LTS^U $\mathcal{S} = (S, (R_a), (U_i), V)$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$

function CheckSE($\mathcal{S}, s, \mathcal{A}$)
  Build product digraph $G = (Q \times S,\ E)$
  $Bad \leftarrow \{(q, t) \in V \mid \exists\, a \in Act : \delta(q, a) \neq \emptyset \text{ and } R_a(t) = \emptyset\}$
  for $q_0 \in I$ **do**
    if DFS((q_0, s)) with target Bad **then return true**
    end if
  end for
  return false
end function

Require: reg-LTS^U $\mathcal{S}$, automaton $\mathcal{A}$, truth set $[\![\varphi]\!]^{\mathcal{S}}$

function CheckCond1($\mathcal{S}, \mathcal{A}, [\![\varphi]\!]^{\mathcal{S}}$)
  for $s \in [\![\varphi]\!]^{\mathcal{S}}$ **do**
    if CheckSE($\mathcal{S}, s, \mathcal{A}$) **then return false**

```
        end if
    end for
    return true
end function
```

In Haskell:

```haskell
-- Compute the set of bad vertices in G.
-- A vertex (q, t) is bad iff there exists an action a such that:
--   (i)  the automaton has a transition from q under a, and
--   (ii) the LTS has no successor from t under a.
badVertices :: RegLTSU -> Automaton -> [GVertex]
badVertices m atmn =
    nub [ (q, t)
        | q <- statesA atmn,
        t <- statesM m,
        a <- actionsA atmn,
        not (null (getAutNext atmn q a)),  --  delta(q,a) != empty
        null (getLtsNext m t a)            --  R_a(t) == empty
        ]

-- checker whether there is a path from (q0, s) to a bad vertex.
-- If so, return True. This means that s not in SE(L(A)).
-- Else, return False. This means that s in SE(L(A)).
checkSE :: RegLTSU -> State -> Automaton -> Bool
checkSE m s atmn =
    let g   = buildDigraph m atmn
        bad = badVertices m atmn
    in  any (\q0 -> dfsReachable g (q0, s) bad []) (initial atmn)

checkCond1 :: RegLTSU -> Automaton -> [State] -> Bool
checkCond1 m atmn =
    all (\s -> not (checkSE m s atmn))
```

Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two automata. To check whether $L(\mathcal{A}_1) \cap L(\mathcal{A}_2) \neq \emptyset$, it suffices to construct the product automaton and check whether any accepting state pair in $F_1 \times F_2$ is reachable from some initial state pair in $I_1 \times I_2$.

Require: Automata $\mathcal{A}_1 = (Q_1, Act_1, \delta_1, I_1, F_1)$ and $\mathcal{A}_2 = (Q_2, Act_2, \delta_2, I_2, F_2)$

function IntersectionNonEmpty($\mathcal{A}_1, \mathcal{A}_2$)
//Build product automaton
    $V \leftarrow Q_1 \times Q_2$
    $E \leftarrow \{((q_1, q_2), (q_1', q_2')) \mid a \in Act_1 \cap Act_2,\ q_1 \xrightarrow{a} q_1' \in \delta_1,\ q_2 \xrightarrow{a} q_2' \in \delta_2\}$
    $G \leftarrow (V, E)$
    $T \leftarrow F_1 \times F_2$
    for $(q_1, q_2) \in I_1 \times I_2$ **do**
        if DFS((q_1, q_2)) with target T **then return true**
        end if
    end for
    return false
end function

To check Condition (2), it suffices to check whether

$$L(\mathcal{A}) \cap \bigcup \{L(\mathcal{A}_{(t_1,t_2)}) \mid t_1 \in [\![\varphi]\!]^{\mathcal{S}},\ t_2 \in [\![\neg\psi]\!]^{\mathcal{S}}\} = \emptyset,$$

where $\mathcal{A}_{(t_1,t_2)} = (S, Act, \delta', \{t_1\}, \{t_2\})$ is the path automaton whose transitions mirror the LTS, i.e. $\delta'(t, a) = \{t' \mid (t, t') \in R_a\}$. Its language is precisely $L(\mathcal{A}_{(t_1,t_2)}) = \{\sigma \in Act^* \mid t_2 \in R_\sigma(t_1)\}$.

By distributivity of $\cap$ over $\bigcup$, this is equivalent to checking that

$$L(\mathcal{A}) \cap L(\mathcal{A}_{(t_1,t_2)}) = \emptyset \quad \text{for all } (t_1, t_2) \in [\![\varphi]\!]^{\mathcal{S}} \times [\![\neg\psi]\!]^{\mathcal{S}}.$$

Require: reg-LTS^U $\mathcal{S}$, automaton $\mathcal{A}$, truth sets $[\![\varphi]\!]^{\mathcal{S}}$ and $[\![\neg\psi]\!]^{\mathcal{S}}$

function CheckCond2($\mathcal{S}, \mathcal{A}, [\![\varphi]\!]^{\mathcal{S}}, [\![\neg\psi]\!]^{\mathcal{S}}$)
  for $t_1 \in [\![\varphi]\!]^{\mathcal{S}}$ **do**
    for $t_2 \in [\![\neg\psi]\!]^{\mathcal{S}}$ **do**
      if IntersectionNonEmpty($\mathcal{A}, \mathcal{A}_{(t_1,t_2)}$) **then return false**
      end if
    end for
  end for
  return true
end function

In Haskell:

```haskell
-- Construct A_(t1, t2)
-- This automaton accepts all plans from t1 to t2
buildPathAutomaton :: RegLTSU -> State -> State -> Automaton
buildPathAutomaton m t1 t2 = ATMN
    { statesA      = statesM m,
    actionsA     = acts,
    transitionsA = [ ((s, a), getLtsNext m s a) | s <- statesM m, a <- acts],
    initial      = [t1],
    final        = [t2]
    }
  where
    -- Collect all actions that appear in the LTS relations
    acts = nub [a | (a, _) <- relationsM m]

-- Check the intersection of L(A1) and L(A2) is empty or not
intersectionNonEmpty :: Automaton -> Automaton -> Bool
intersectionNonEmpty a1 a2 =
    any (\sv -> dfsReachable prodGraph sv targets []) startVs
  where
    -- Shared actions
    sharedActs = nub [a | a <- actionsA a1, a `elem` actionsA a2]

    -- Product graph vertices and edges
    prodNodes = [(q1, q2) | q1 <- statesA a1, q2 <- statesA a2]
    prodEdges = [ ((q1, q2), (q1', q2'))
                | (q1, q2) <- prodNodes,
                a          <- sharedActs,
                q1'        <- getAutNext a1 q1 a,
                q2'        <- getAutNext a2 q2 a
                ]
    prodGraph = Digraph prodNodes prodEdges

    -- Start from any pair of initial states
    startVs = [(q1, q2) | q1 <- initial a1, q2 <- initial a2]

    -- Target: any pair of accepting states
    targets = [(q1, q2) | q1 <- final a1, q2 <- final a2]

-- Check for each A: for all (t1,t2): L(A) \cap L(A_(t1, t2)) = empty?
-- We check the emptiness of the intersection by checking the reachability of the product graph(
     automaton)
-- If empty, then cond 2 is SAT
checkCond2 :: RegLTSU -> Automaton -> [State] -> [State] -> Bool
checkCond2 m aut phiStates negPsiStates = not (any violates pairs)
  where
    pairs = [(t1, t2) | t1 <- phiStates, t2 <- negPsiStates]
    -- Check the intersection. Non-empty => cond 2 is UNSAT.
    violates (t1, t2) =
        let pathAut = buildPathAutomaton m t1 t2
        in  intersectionNonEmpty aut pathAut
```

Following the semantics defined in Definition 4.7 and putting things together, we complete the model checker for $reg\text{-}\mathcal{L}^{U}_{KH}$.

```
-- [[phi]]= set of states that phi holds
truthSet :: RegLTSU -> RegForm -> [State]
truthSet m f = [s | s <- statesM m, isTrueReg (m, s) f]

-- Satisfaction relation for the propositional fragment
isTrueReg :: (RegLTSU, State) -> RegForm -> Bool
isTrueReg (m, s) (Prop p) =
    case lookup s (valuationM m) of
        Just props -> p `elem` props
        Nothing -> False
isTrueReg (m, s) (Not f) =
    not (isTrueReg (m, s) f)
isTrueReg (m, s) (Disj f g) =
    isTrueReg (m, s) f || isTrueReg (m, s) g

-- Kh_a(phi, psi) holds iff there exists A in U_a such that
-- (1) [[phi]] is subset of SE(L(A))
-- (2) R_{L(A)}([[phi]]) is subset of [[psi]]
isTrueReg (m, _) (KHI agent phi psi) =
    any (\aut -> checkCond1 m aut phiStates
              && checkCond2 m aut phiStates negPsiStates
    ) (getAgentAuts m agent)
  where
    phiStates    = truthSet m phi          -- [[phi]]
    negPsiStates = truthSet m (Not psi)    -- [[neg psi]]

-- Infix alias for the satisfaction relation
(||=) :: (RegLTSU, State) -> RegForm -> Bool
(||=) = isTrueReg
```

3.4 Parsing for $reg\text{-}\mathcal{L}^{U}_{KH}$

Formulas may be created in ghci using `parseForm`. The following inputs are accepted.

- 'p' or 'P' followed by an integer n returns `P n`
- '!' followed by a valid input p returns `Neg p`
- 'v' or 'V' prefixed and followed by valid inputs p and q returns `Disj p q`
- "KH" followed by index i valid inputs p and q returns `KH i p q`
- "->" prefixed and followed by valid inputs p and q returns `Disj (Not p) q` (abbreviation)
- '^' prefixed and followed by valid inputs p and q returns `Not (Disj (Not p) (Not q))` (abbreviation)

```
pRegForm :: Parsec String () RegForm
pRegForm = spaces *> pImpl where
    -- Start parsing at implication level
    -- Abbreviation: Implication (right-associative)
    pImpl = chainr1 pDisj (spaces *> string "->" *> spaces >> return (Disj . Not))

    -- Disjunction (left-associative)
    pDisj = chainl1 pConj (spaces *> oneOf "vV" *> spaces >> return Disj)

    -- Abbreviation: Conjunction (left-associative)
    pConj = chainl1 pPrefix (spaces *> char '^' *> spaces >> return (\p q -> Not (Disj (Not p) (Not
        q))))
```

```
    pPrefix = try pNeg
        <|> try pKH
        <|> pRemainder

    -- Negation
    pNeg = char '!' >> spaces >> Not <$> pPrefix

    -- Knowing How for Agent
    pKH = KHI <$> (string "KH" >> spaces *> pAgent)
        <*> (spaces *> pPrefix)
        <*> (spaces *> pPrefix)
    pAgent = read <$> many1 digit

    pRemainder = pVar <|> between (char '(' *> spaces) (spaces *> char ')') pRegForm
    pVar = spaces *> oneOf "pP" *> spaces *> (Prop . read <$> many1 digit) <* spaces



parseRegForm :: String -> Either ParseError RegForm
parseRegForm = parse (pRegForm <* eof) "input"

evalRegForm :: (RegLTSU, State) -> String -> Bool
evalRegForm (m, s) str =
    case parseRegForm str of
        Right f -> isTrueReg (m, s) f
        Left _  -> error "Invalid formula"
```

3.5 Random Generation for $reg\text{-}\mathcal{L}^U_{KH}$

3.5.1 Model Generation with Parameters

To facilitate testing in the multi-agent setting, we provide a function that generates a random reg-LTS^U model with a fixed number of states, propositions, actions, and agents.

The generated model includes:

- a finite set of states $\{1, \ldots, n\}$,
- a set of actions $\{1, \ldots, k\}$,
- a valuation assigning each state a random subset of propositions $\{1, \ldots, m\}$,
- a family of action-labelled transitions,
- and for each agent, a set of automata representing uncertainty over plans.

```
-- Generate a random reg-LTS^U model with parameters
-- n: number of states
-- m: number of propositions
-- k: number of actions
-- a: number of agents
generateRegLTSU :: Int -> Int -> Int -> Int -> Gen RegLTSU
generateRegLTSU n m k numAgents = do
    let n' = max 1 n
    let m' = max 0 m
    let k' = max 0 k
    let a' = max 1 numAgents

    let sts = [1 .. n']
    let acts = [1 .. fromIntegral k']
    let props = [1 .. m']
```

```haskell
    -- Generate transitions
    rels <- mapM (\_ -> generateRandomRel sts) acts

    -- Generate valuations
    vals <- mapM (generateRandomValuation props) sts

    -- Generate uncertainty (automata per agent)
    auts <- mapM (\_ -> generateAutomata sts acts) [1 .. a']

    return $ RegLTSU sts (zip acts rels) (zip [1..] auts) vals


-- Generate a random relation (same style as single-agent)
generateRandomRel :: [State] -> Gen Rel
generateRandomRel sts = do
    numEdges <- choose (0, length sts * length sts)
    edges <- vectorOf numEdges $ do
        u <- elements sts
        v <- elements sts
        return (u, v)
    return (nub edges)


-- Generate valuation for a state
generateRandomValuation :: [Proposition] -> State -> Gen (State, [Proposition])
generateRandomValuation props s = do
    ps <- sublistOf props
    return (s, ps)


-- Generate a list of automata for one agent
generateAutomata :: [State] -> [Action] -> Gen [Automaton]
generateAutomata sts acts = do
    numAuts <- choose (1, 3)
    vectorOf numAuts (generateAutomaton sts acts)


-- Generate a simple random automaton
generateAutomaton :: [State] -> [Action] -> Gen Automaton
generateAutomaton sts acts = do
    let qs = sts

    trans <- sequence
        [ do next <- sublistOf qs
             return ((q, a), next)
        | q <- qs, a <- acts
        ]

    initStates <- sublistOf qs
    finalStates <- sublistOf qs

    return $ ATMN
        { statesA = qs
        , actionsA = acts
        , transitionsA = trans
        , initial = if null initStates then [head qs] else initStates
        , final = if null finalStates then [head qs] else finalStates
        }
```

It is possible to generate models and test formulas interactively in `ghci`. For instance:

```
ghci> m <- generate (generateRegLTSU 5 3 2 2)
ghci> m
RegLTSU {statesM = [1,2,3,4,5],
relationsM = [(1,[(1,1),(3,1),(4,2)]),
(2,[(4,4),(5,4),(1,1),(1,2),(3,2),
(1,3),(4,3),(5,3),(1,4),(5,1),(3,1),
(2,3),(2,1),(1,5),(2,4),(4,2)])],
```

```
uncertainty = [(1,[ATMN {statesA = [1,2,3,4,5], actionsA = [1,2],
transitionsA = [((1,1),[2,3]),((1,2),[1,3,5]),((2,1),[1,3,5]),
((2,2),[1,2,3]),((3,1),[2,4]),((3,2),[3,4]),
((4,1),[1,3]),((4,2),[2,4,5]),((5,1),[4]),((5,2),[3])],
initial = [3], final = [3,5]}]),
(2,[ATMN {statesA = [1,2,3,4,5], actionsA = [1,2],
transitionsA = [((1,1),[3,5]),
((1,2),[1,2,5]),((2,1),[1,3,5]),((2,2),[1,3,4,5]),
((3,1),[2,5]),((3,2),[5]),((4,1),[1,3]),
((4,2),[3,4,5]),((5,1),[3]),((5,2),[3])],
initial = [2,5], final = [1,3,5]}])],
valuationM = [(1,[1,3]),(2,[1,3]),(3,[1]),(4,[1,2,3]),(5,[1,3])]}

-- Evaluate a formula at a state
ghci> isTrueReg (m, 1) (KHI 1 (Prop 1) (Prop 2))
False

-- Using the parser
ghci> evalRegForm (m, 1) "KH1 p1 p2"
False
```

In the example above, `generateRegLTSU 5 3 2 2` produces a model with five states, three propositions, two actions, and two agents. Each agent is associated with a randomly generated set of automata representing its uncertainty over plans.

3.5.2 Arbitrary Instances for QuickCheck

To support property-based testing, we define generators for both formulas and models in the multi-agent setting.

Since the structure of formulas is independent of any particular model, the generator is defined separately from model generation. However, to avoid mismatches between formulas and models, we parameterize the generator by the number of agents. This ensures that all occurrences of the modality Kh_i refer only to valid agent indices.

```
-- Generate a random RegForm with bounded size and a fixed number of agents
generateRegForm :: Int -> Int -> Gen RegForm
generateRegForm numAgents = randomRegForm
  where
    numAgents' = max 1 numAgents

    randomRegForm :: Int -> Gen RegForm
    randomRegForm 0 = Prop <$> choose (1, 5)
    randomRegForm n = oneof
        [ Prop <$> choose (1, 5)
        , Not <$> randomRegForm (n - 1)
        , Disj <$> randomRegForm (n `div` 2) <*> randomRegForm (n `div` 2)
        , KHI <$> choose (1, numAgents')
              <*> randomRegForm (n `div` 2)
              <*> randomRegForm (n `div` 2)
        ]
```

4 Testing and Validation

We now use the library QuickCheck to randomly generate models and states in order to test central semantic properties of both the basic single-agent logic $\mathcal{L}_{Kh}$ and the multi-agent regular plan logic reg-$\mathcal{L}^{U}_{KH}$.

```
module Main where

import Test.QuickCheck
import Test.Hspec
import Data.List.NonEmpty (toList)

import SingleAgent
import MultiAgent
```

The following uses the HSpec library to define our test suite.

To perform these tests, we first define auxiliary types `PointedModel` (for single-agent models) and `PointedRegModel` (for multi-agent models), together with their `Arbitrary` instances. This allows QuickCheck to select a random evaluation point inside a randomly generated model.

```
-- Pointed model for single-agent L_Kh
newtype PointedModel = PM (AbilityMap, State) deriving (Show)

instance Arbitrary PointedModel where
    arbitrary = do
        m <- arbitrary
        s <- elements (toList (states m))
        return (PM (m, s))

-- Pointed model for multi-agent reg-LTS^U
newtype PointedRegModel = PRM (RegLTSU, State) deriving (Show)

instance Arbitrary PointedRegModel where
    arbitrary = do
        nStates <- chooseInt (1, 3)
        nProps  <- chooseInt (1, 3)
        nActs   <- chooseInt (1, 3)
        nAgents <- chooseInt (1, 3)
        m <- generateRegLTSU nStates nProps nActs nAgents
        s <- elements (statesM m)
        return (PRM (m, s))
```

4.1 Tests for $\mathcal{L}_{Kh}$

We first test some basic properties of the single-agent logic.

The first group concerns the Boolean fragment.

- **Double Negation Elimination**
  - $\neg\neg\varphi \equiv \varphi$
- **Commutativity of Conjunction**
  - $\varphi \wedge \psi \equiv \psi \wedge \varphi$

- **Identity of Conjunction**
  - $\varphi \wedge \top \equiv \varphi$

We then test a property specific to Kh. We verify that $Kh(\psi, \varphi)$ is *global*: its truth value does not depend on the current evaluation state, since its semantic clause quantifies over all states satisfying the precondition ψ. Therefore, if $Kh(\psi, \varphi)$ holds at one state of a model, it should hold at every state of that model.

```
-- Helper: check if two formulas are equivalent at a specific point
isEquivalent :: (AbilityMap, State) -> Form -> Form -> Bool
isEquivalent (m, s) f1 f2 = isTrue (m, s) f1 == isTrue (m, s) f2
```

4.2 Tests for $reg\text{-}\mathcal{L}^{U}_{KH}$

In the multi-agent setting, we test several central semantic properties of the Kh_i-modality.

We test the case of an **impossible precondition**. If the precondition is contradictory, then no states satisfy it, and both semantic conditions of $Kh_i(\psi, \varphi)$ are trivially satisfied. Hence, $Kh_i(\psi, \varphi)$ should hold vacuously in this case.

As in the single-agent case, we verify that $Kh_i(\psi, \varphi)$ is **global** in the model: its truth value does not depend on the evaluation state, since its semantics quantifies over all states satisfying the precondition.

Finally, we include a test at the level of the underlying automata machinery. In particular, we verify that the check for non-emptiness of language intersection between automata is symmetric.

```
-- Helper: universal implication in multi-agent models
isUniversallyImpliedReg :: RegLTSU -> RegForm -> RegForm -> Bool
isUniversallyImpliedReg m psi phi =
    all (\s -> not (isTrueReg (m, s) psi) || isTrueReg (m, s) phi) (statesM m)

-- A contradictory formula, used as bottom
bottomReg :: RegForm
bottomReg = Not (Disj (Prop 1) (Not (Prop 1)))

-- Helper: number of agents in a reg-LTS^U model, used to ensure that generated formulas only refer
     to valid agent indices
numAgentsIn :: RegLTSU -> Int
numAgentsIn m = length (uncertainty m)
```

4.3 HSpec Test Suite

The test suite below combines general Boolean checks with semantic properties that are characteristic of knowing-how logics.

```
main :: IO ()
main = hspec $ do

  describe "Single-Agent Logic L_Kh" $ do

    it "Boolean: Double Negation Elimination (!!f <=> f)" $
```

```
      property $ \(PM (m, s)) f ->
        isEquivalent (m, s) (Neg (Neg f)) f

    it "Boolean: Commutativity of Conjunction (f1 ^ f2 <=> f2 ^ f1)" $
      property $ \(PM (m, s)) f1 f2 ->
        isEquivalent (m, s) (Conj f1 f2) (Conj f2 f1)

    it "Boolean: Identity of Conjunction (f ^ T <=> f)" $
      property $ \(PM (m, s)) f ->
        isTrue (m, s) (Conj f T) === isTrue (m, s) f

    it "KH is global: truth does not vary between states" $
      property $ \m f g ->
        let sts = toList (states m)
            results = [ isTrue (m, s) (KH f g) | s <- sts ]
        in not (null results) ==> all (== head results) results

  describe "Multi-Agent Logic reg-L^U_KH" $ do



    it "Vacuous Precondition: knowing how from contradiction holds vacuously" $
      property $ \(PRM (m, s)) agent ->
        let nA = numAgentsIn m
        in forAll (sized (generateRegForm nA)) $ \phi ->
           let aIdx = (agent `mod` nA) + 1
           in isTrueReg (m, s) (KHI aIdx bottomReg phi)

    it "KH_i is global: agent ability is a model-wide property" $
      property $ \(PRM (m, _)) agent ->
        let nA = numAgentsIn m
        in forAll (sized (generateRegForm nA)) $ \f ->
           forAll (sized (generateRegForm nA)) $ \g ->
             let aIdx = (agent `mod` nA) + 1
                 sts = statesM m
                 results = [ isTrueReg (m, s) (KHI aIdx f g) | s <- sts ]
             in not (null results) ==> all (== head results) results

    it "Intersection checking is symmetric: L(A1) intersect L(A2) is non-empty iff L(A2) intersect L
         (A1) is non-empty" $
      property $ \(PRM (m, _)) agent1 agent2 ->
        let nA    = numAgentsIn m
            aIdx1 = (agent1 `mod` nA) + 1
            aIdx2 = (agent2 `mod` nA) + 1
            auts1 = getAgentAuts m aIdx1
            auts2 = getAgentAuts m aIdx2
        in not (null auts1) && not (null auts2) ==>
           forAll (elements auts1) $ \aut1 ->
           forAll (elements auts2) $ \aut2 ->
             intersectionNonEmpty aut1 aut2
             ==
             intersectionNonEmpty aut2 aut1
```

Running `stack test` yields the following relevant test output.

```
report> test (suite: quickcheck-logic)


Single-Agent Logic L_Kh
  Boolean: Double Negation Elimination (!!f <=> f)
    +++ OK, passed 100 tests.
  Boolean: Commutativity of Conjunction (f1 ^ f2 <=> f2 ^ f1)
    +++ OK, passed 100 tests.
  Boolean: Identity of Conjunction (f ^ T <=> f)
    +++ OK, passed 100 tests.
  KH is global: truth does not vary between states
```

```
    +++ OK, passed 100 tests.
Multi-Agent Logic reg-L^U_KH
  Vacuous Precondition:
  knowing how from contradiction holds vacuously
    +++ OK, passed 100 tests.
  KH_i is global: agent ability is a model-wide property
    +++ OK, passed 100 tests.
  Intersection checking is symmetric:
  L(A1) intersect L(A2) is non-empty
  iff L(A2) intersect L(A1) is non-empty

Finished in 5.2967 seconds
7 examples, 0 failures
```

Overall, these results provide evidence that the implementation captures several expected semantic properties.

5 Conclusion

In this report, we presented concrete Haskell implementations of two knowing-how logics: the basic logic $\mathcal{L}_{Kh}$ and its extension with regularity constraints reg-$\mathcal{L}^U_{KH}$. We also implemented an explicit model checker with bounded plan length for $\mathcal{L}_{Kh}$, as well as a model checker for reg-$\mathcal{L}^U_{KH}$ based on the procedure described in [2]. Since the original paper does not spell out all technical details of the reachability tests, we made these constructions explicit in our implementation by using depth-first search, which satisfies the time bound. We also incorporated a small optimisation in the implementation of the procedure `CheckSE`: instead of performing reachability tests inside nested loops, we first compute the set of bad points and then perform the reachability checks. This avoids repeated reachability computations. To support experimentation, we provided parsers for both logics, as well as generators for random models and formulas. These components enabled systematic validation using QuickCheck and HSpec.

We note that the current implementation assumes that all automata provided in the uncertainty component are well-formed. In particular, we do not explicitly verify whether they satisfy the structural conditions required by the theoretical framework. As a result, it is the responsibility of the user to ensure that the automata used in the model conform to these requirements. A natural direction for future work would therefore be to implement a dedicated sanity checker for automata.

References

[1] C. Areces, R. Fervari, A. R. Saravia, and F. R. Velázquez-Quesada. Uncertainty-based semantics for multi-agent knowing how logics. In *Proceedings of the 18th Conference on Theoretical Aspects of Rationality and Knowledge (TARK 2021)*, volume 335 of *EPTCS*, pages 23–37, 2021.

[2] S. Demri and R. Fervari. Model-checking for ability-based logics with constrained plans. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI-23)*, volume 37, pages 6305–6312, 2023.

[3] Y. Wang. A logic of knowing how. In *Logical Reasoning and Interaction (LORI'15)*, volume 9394 of *Lecture Notes in Computer Science (LNCS)*, pages 392–405. Springer, 2015.